

VLA/VA 模型长程操控基准测评技术文档

基于 StarVLA 的 CALVIN ABC→D Action Policy Model

队伍：【待填写】 姓名：【待填写】

代码仓库：【待填写：GitHub / Gitee / 内部仓库链接】 模型权重：【待填写：HuggingFace 链接或下载链接】

文档说明：本文档参考 技术文档-*demo.pdf* 的正文结构组织，覆盖技术路线选择论证、VLA/WM 路线对比、CALVIN ABC→D 测评结果、Failure Pattern 分析、后训练/改进方案、工程实现与可复现性、个人贡献说明和交付物链接。所有需要根据真实实验结果补齐的内容均以【待填写】标出。

目录

1	项目概述	3
2	模型与技术路线选择	4
2.1	技术路线选择	4
2.2	选择 VLA 路线的原因	4
2.3	与 WM 路线的对比	4
3	数据与训练设置	5
3.1	数据来源	5
3.2	训练流程	5
3.2.1	第一阶段：公开数据预训练	5
3.2.2	第二阶段：CALVIN ABC 监督微调	5
3.2.3	第三阶段：Failure-Aware Post-Training	6
4	CALVIN ABC 到 D 测评结果	6
4.1	主结果	6
4.2	低成功率任务统计	6
5	Failure Pattern 分析	7
5.1	Failure logging 设计	7
5.2	失败模式分类	7
5.2.1	链式误差累积	8
5.2.2	动作精度不足	8
5.2.3	夹爪时机错误	8
5.2.4	环境 D 泛化失败	8
6	Failure-Aware Post-Training 方法	8
6.1	方法动机	8
6.2	Failure-Aware Sampling	9
6.3	Weighted Action Loss	9
6.4	Action Chunk Time Weighting	9
6.5	后训练默认配置	10
7	Push-Aware 改进方案	10
7.1	问题发现	10
7.2	Push 任务失败根因分析	10
7.3	Push-Aware Sampling	11

7.4	Push-Focused Weighted Loss	11
7.5	两阶段 Post-Training 设计	11
8	消融实验设计	12
9	工程实现	12
9.1	代码修改概述	12
9.2	Failure Logging	13
9.3	Failure-Aware Dataloader	13
9.4	Weighted Flow-Matching Loss	13
9.5	Post-Training Script	13
10	可复现性说明	14
10.1	训练环境	14
10.2	数据路径	14
10.3	训练命令	14
11	实验结果分析	15
11.1	整体结果分析	15
11.2	Push 类任务结果分析	15
11.3	失败案例分析	16
12	局限性与未来改进	16
13	结论	16
14	个人贡献说明	17
15	交付物链接	17
A	仍需补充的信息清单	18

1 项目概述

本项目基于 StarVLA 框架，在 CALVIN ABC→D 长程机器人操控基准上实现并评测一个 Action Policy Model。CALVIN ABC→D 要求模型在训练环境 A/B/C 上学习机器人操作策略，并在未见过的环境 D 中连续完成 5 个子任务链，因此该任务重点考察模型的跨环境泛化能力、长程链式执行能力以及对误差累积的鲁棒性。

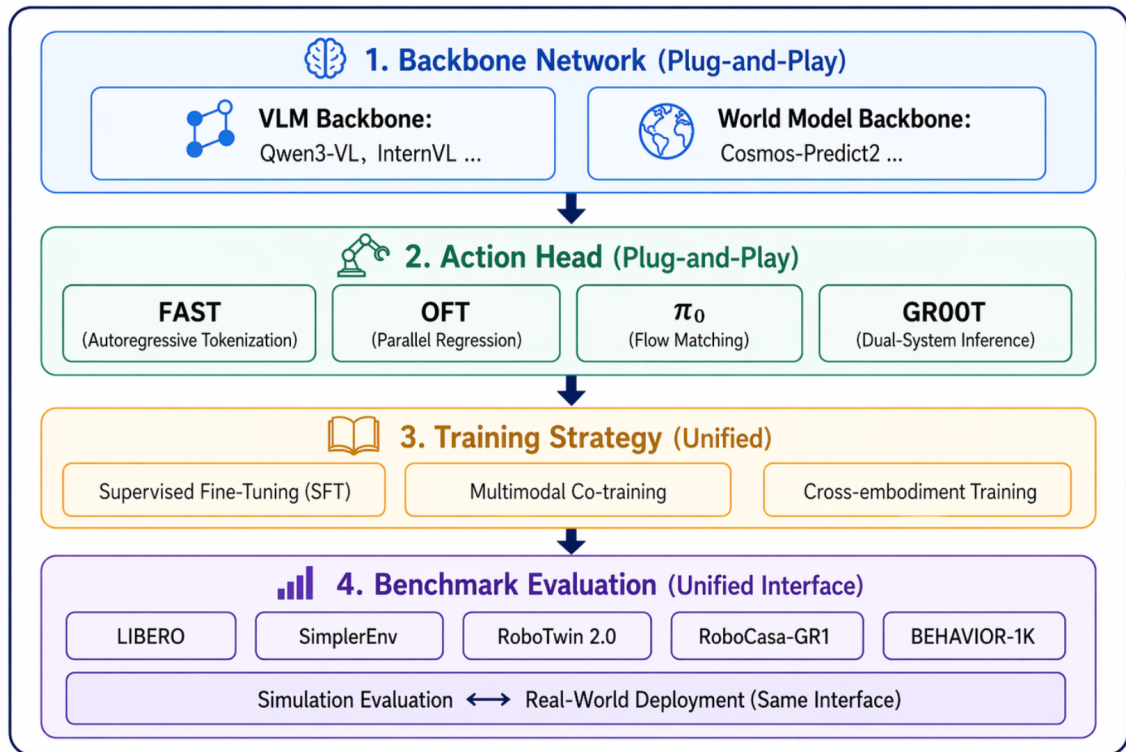
本项目采用 Vision-Language-Action (VLA) 路线，选择 Qwen3.5-4B + PI / LayerwiseFM / Flow-Matching Action Head 作为基础模型架构，并在公开数据预训练、CALVIN ABC 监督微调的基础上，引入 Failure-Aware Post-Training 策略，对评测过程中暴露出的低成功率任务和典型失败模式进行针对性优化。

整体训练与评测流程如下：

Listing 1: 整体训练与评测闭环

```
Public / LIBERO Pretraining
↓
CALVIN ABC Supervised Finetuning
↓
CALVIN D Evaluation
↓
Failure Pattern Mining
↓
Failure-Aware Post-Training
↓
CALVIN D Re-evaluation
```

本项目的核心目标不是仅提升单步动作预测精度，而是提升模型在 CALVIN ABC→D 长程任务链中的平均链长和 Task 1–Task 5 阶段成功率，尤其关注模型在未见环境 D 中的稳定执行能力。



Four Plug-and-Play Layers: Backbone Network → Action Head → Training Strategy → Benchmark Evaluation

图 1: StarVLA 模块化框架。本文档中的模型、训练、评测和部署均复用 StarVLA 的统一接口。

2 模型与技术路线选择

2.1 技术路线选择

本项目选择 VLA 路线，而不是 World Model (WM) 路线。VLA 模型直接从视觉观察、语言指令和机器人状态中预测低层连续动作，具有训练流程直接、推理开销较低、与 StarVLA 现有 CALVIN 工具链兼容性较好的优点。

本项目采用的模型配置如下：

表 1: 模型与实验基础配置

项目	设置
视觉-语言骨干模型	Qwen3.5-4B
动作头架构	PI / LayerwiseFM / Flow-Matching Action Head
训练数据	【待填写：例如 LIBERO + CALVIN ABC】
评测基准	CALVIN ABC→D
训练设备	【待填写：例如 8 × H200】
训练总步数	pretrain 【待填写：例如 20000 / 80000】 + finetune 30000 + post-train 5000

在 StarVLA 支持的动作头中，PI / LayerwiseFM 使用 Flow-Matching 扩散式动作头，一次预测一个 action chunk。相比单步动作预测，该方式可以建模短期轨迹结构，有利于机器人在连续控制中保持动作平滑性和局部时序一致性。因此，本项目选择该类动作头作为主要策略模型。

2.2 选择 VLA 路线的原因

数据效率。VLA 路线可以直接利用已有的机器人轨迹数据进行监督学习，训练目标清晰，工程实现成本较低。对于两天实训任务而言，VLA 路线更适合快速完成从数据准备、监督微调、评测到失败分析的完整闭环。相比之下，WM 路线通常需要额外学习环境状态转移或视频生成模型，对数据规模、训练稳定性和算力要求更高。在有限时间内，完整训练或充分调优一个 World Model 的风险较大。

泛化能力。CALVIN ABC→D 需要模型从训练环境 A/B/C 泛化到未见过的 D 环境。VLA 模型依赖大规模视觉-语言模型作为表征骨干，可以继承预训练模型中的视觉语义理解能力，对背景、光照、物体外观变化具有一定鲁棒性。本项目进一步通过公开数据预训练和 CALVIN ABC 监督微调，使模型同时具备通用视觉语言表征和目标任务相关的低层动作能力。

长程规划能力。VLA 路线本身并不显式建模完整长程世界状态，因此在长任务链中容易出现误差累积。为缓解这一问题，本项目没有简单增加训练步数，而是引入评测驱动的 Failure-Aware Post-Training：先在完整 CALVIN D 评测中挖掘失败模式，再对高频失败任务进行重采样和加权训练，从而使训练目标更贴近长程评测中的真实问题。

计算成本与推理效率。VLA 策略在推理时直接输出动作，不需要显式进行未来视频生成或多步世界状态预测，因此推理成本相对较低，更适合 CALVIN 长程 rollout 评测。在本项目的硬件条件下，VLA 路线能够在较短时间内完成多轮训练与评测迭代。

2.3 与 WM 路线的对比

World Model 路线的优势在于能够显式建模未来状态变化，理论上更适合长程规划、遮挡场景推理和反事实预测。对于需要复杂物理交互或长期状态预测的任务，WM 可能具备更强的可解释性和规划潜力。

但在本项目中，WM 路线存在以下限制：

1. 训练成本更高，需要学习视觉状态转移或未来视频生成。
2. 对数据规模和数据质量要求更高。

- 3. 推理阶段可能需要额外的 rollout、采样或规划过程，计算开销更大。
- 4. 两天实训时间内完整验证 WM 路线的风险较高。

因此,本项目选择 VLA 作为主路线,并将主要创新点放在长程失败模式挖掘和 Failure-Aware Post-Training 上, 以更低工程风险完成可复现的实验闭环。

3 数据与训练设置

3.1 数据来源

本项目使用的数据包括：

表 2: 数据来源

数据类型	设置
公开预训练数据	【待填写：例如 LIBERO / OXE / Bridge / RoboCasa / RoboTwin】
CALVIN 训练数据	CALVIN ABC split
CALVIN 评测数据	CALVIN D split

其中, CALVIN ABC 用于监督微调, CALVIN D 用于跨环境泛化评测。ABC→D split 能够有效检验模型是否真正学到了可迁移的操作策略, 而不是仅记忆训练环境中的视觉分布。

3.2 训练流程

本项目训练流程分为三个阶段。

3.2.1 第一阶段：公开数据预训练

在公开机器人操作数据集上进行预训练，使模型获得基本的视觉-语言-动作对齐能力和通用操作先验。

表 3: 预训练配置

项目	设置
预训练数据集	【待填写】
预训练步数	【待填写：例如 20000 / 80000】
预训练 checkpoint	【待填写】

3.2.2 第二阶段：CALVIN ABC 监督微调

使用 CALVIN ABC 训练集进行行为克隆式监督微调，使模型适配 CALVIN 的动作空间、任务语言和环境交互形式。

表 4: CALVIN 微调配置

项目	设置
微调数据	CALVIN ABC
微调步数	30000
学习率	【待填写：例如 base 2.5e-5, action 1e-4】
batch size	【待填写】
冻结模块	【待填写：例如 qwen_vl_interface】
微调 checkpoint	【待填写】

3.2.3 第三阶段：Failure-Aware Post-Training

在常规监督微调之后，本项目加入轻量的 Failure-Aware Post-Training。该阶段利用 CALVIN D 评测中暴露出的失败模式，对模型进行短程、有针对性的后训练。

表 5: 后训练配置

项目	设置
后训练 warm-start checkpoint	【待填写】
后训练步数	5000
action head learning rate	5e-5
failure-aware weight	2.0
top-k failure instructions	20
冻结模块	【待填写：例如 qwen_vl_interface】

4 CALVIN ABC 到 D 测评结果

本项目在 CALVIN ABC→D split 上进行完整评测，记录 Task 1 到 Task 5 各阶段成功率和平均任务链长度。

4.1 主结果

表 6: CALVIN ABC→D 主结果

模型 / 设置	Task 1	Task 2	Task 3	Task 4	Task 5	Avg. Chain
CALVIN finetune baseline	【待填】	【待填】	【待填】	【待填】	【待填】	【待填】
Failure-Aware Post-Training	【待填】	【待填】	【待填】	【待填】	【待填】	【待填】
Push-Aware Post-Training	【待填】	【待填】	【待填】	【待填】	【待填】	【待填】
Final model	【待填】	【待填】	【待填】	【待填】	【待填】	【待填】

从评测结果可以观察到，【待填写：简述 Task 1–5 成功率变化，例如 Task 1/2 较高，Task 3–5 下降明显】。这表明模型在短链任务中已经具备一定执行能力，但在长程连续任务中仍受到状态偏移和误差累积影响。

4.2 低成功率任务统计

根据 per-task success rate 统计，当前模型成功率最低的任务主要集中在以下类别：

表 7: 低成功率任务示例

任务名称	成功率
push_blue_block_right	19.4%
push_red_block_left	24.4%
push_pink_block_left	26.5%
push_red_block_right	36.4%
push_pink_block_right	41.2%
push_blue_block_left	59.0%

从结果可以看出，失败并非均匀分布在所有 CALVIN 子任务上，而是明显集中在 push-block left/right 这一类横向推动任务中。相比之下，push_into_drawer 的成功率仍可达到 84.4%，说明模型不是完全不会执行 push 动作，而是在 free block lateral pushing 场景中存在明显短板。

该现象说明，模型在无约束物体横向推动中可能存在以下问题：

- 1. 对 block 的接触位置不稳定。
- 2. 水平方向位移幅度或方向预测不准确。
- 3. 推动过程中难以保持持续接触。
- 4. 环境 D 中物体颜色、位置或视觉分布变化导致动作泛化不足。

5 Failure Pattern 分析

5.1 Failure logging 设计

为分析模型在 CALVIN D 环境中的失败模式，本项目在评测脚本中加入 failure logging 机制。每当模型在某条任务链中首次失败时，系统会记录该失败案例的细粒度信息，并保存为 failure_log.jsonl。

每条 failure log 包含以下字段：

Listing 2: failure log 字段

```
{
  "sequence_i": "评测序列编号",
  "subtask_i": "失败子任务在任务链中的位置",
  "stage": "失败阶段，对应 Task 1~Task 5",
  "subtask": "失败子任务名称",
  "eval_sequence": "完整任务链",
  "lang_annotation": "失败任务对应语言指令",
  "steps": "rollout 执行步数",
  "reason": "失败原因",
  "gif_path": "debug 模式下失败视频路径"
}
```

此外，评测脚本会生成 failure_summary.json，统计平均链长、各阶段成功率、不同阶段失败次数和不同子任务失败次数。

5.2 失败模式分类

根据 CALVIN D 评测结果，本项目将主要失败模式分为四类。

5.2.1 链式误差累积

模型在 Task 1 或 Task 2 中通常能够完成任务，但随着任务链推进，前序动作中的轻微误差会改变后续任务的初始状态，使模型逐渐偏离训练分布。该问题在 Task 3–Task 5 中更加明显。

典型表现包括：

1. 前一个任务完成位置不够精确，导致后续物体位置偏移。
2. 机器人末端执行器停留在不利姿态，影响下一任务起始动作。
3. 多个小误差在长程 rollout 中逐步累积，最终导致 oracle 条件无法触发。

对应改进方案包括：

1. 使用 action chunk time weighting，提高后半段动作的 loss 权重。
2. 通过 failure-aware sampling 增加长链后段高频失败任务的训练占比。
3. 对 Task 3–Task 5 的失败案例进行单独统计，分析是否存在阶段性退化。

5.2.2 动作精度不足

部分失败案例中，模型能够大致移动到目标附近，但末端执行器位置、姿态或位移幅度不够准确，导致任务 oracle 条件无法触发。该问题在 block pushing、slider、drawer 等任务中较为明显。

典型表现包括接近目标但没有触发成功判定、推动物体距离不足、末端执行器与物体接触点偏移、动作方向正确但幅度不足或过大。对应改进方案包括在 action loss 中提高关键动作维度权重、对低成功率任务进行 task-aware oversampling，并使用 per-task success rate 定位动作原语级别的短板。

5.2.3 夹爪时机错误

抓取、释放、拉动等任务对 gripper 开合时机非常敏感。如果 gripper 过早闭合、过晚释放或方向预测错误，会导致任务失败。该问题可能影响 lift、place、drawer 等任务。

典型表现包括抓取物体前 gripper 已提前闭合、释放位置正确但 gripper 没有及时打开、拉动 drawer 时接触不稳定、grripper 输出在连续动作中发生抖动。对应改进方案包括在通用 failure-aware post-training 中提高 gripper 维度 loss 权重、保持较短训练步数以避免破坏已有基础操作能力，并对涉及 gripper 的任务单独统计失败率。

5.2.4 环境 D 泛化失败

CALVIN ABC→D 的训练环境为 A/B/C，评测环境为 D。环境 D 中的背景、光照、物体布局和视觉分布可能与训练环境存在差异，导致视觉表征和动作预测出现偏移。

典型表现包括模型识别目标物体不稳定、对颜色或物体位置变化敏感、在训练环境中成功率较高但迁移到 D 环境后下降明显。对应改进方案包括引入视觉增强策略，例如颜色扰动、亮度扰动、随机裁剪等；采用更多跨环境公开数据进行预训练；冻结视觉语言 backbone，仅针对 action head 做轻量后训练，降低过拟合风险。

6 Failure-Aware Post-Training 方法

6.1 方法动机

标准监督微调通常基于训练集中的离线轨迹进行行为克隆，优化的是平均动作预测误差。但 CALVIN ABC→D 的难点在于长程闭环执行：模型在 Task 1 中产生的小误差会改变 Task 2 的初始状态，后续任务进一步偏离训练分布，最终导致链式成功率下降。

因此，仅依赖标准监督微调可能不足以优化 Task 3–Task 5 的长程执行表现。本项目采用评测驱动的后训练流程：

Listing 3: 评测驱动后训练流程

```
Evaluate -> Mine Failures -> Reweight Data/Loss -> Post-train -> Re-evaluate
```

该方法将 CALVIN 评测中真实暴露的失败案例转化为训练信号，以降低工程成本提升模型在长程任务中的稳定性。

6.2 Failure-Aware Sampling

评测阶段产生的 `failure_log.jsonl` 包含失败子任务对应的语言指令。后训练时，`dataloader` 会读取该日志，统计出现频率最高的失败指令，并提高这些 `hard instructions` 在训练 `batch` 中的相对占比。

具体实现上，本项目没有直接复制失败样本，也没有改变数据格式，而是对非失败指令样本进行轻量降采样，使失败指令在训练过程中更频繁出现。该设计能够避免过度复制少量失败样本导致过拟合，同时让模型在后训练阶段更集中地修正常见失败任务。

表 8: Failure-Aware Sampling 默认设置

项目	设置
failure-aware weight	2.0
top-k failure instructions	20
failure log path	【待填写】

6.3 Weighted Action Loss

CALVIN 动作维度为：

Listing 4: CALVIN action dimension

```
[x, y, z, roll, pitch, yaw, gripper]
```

在长程操控中，平移动作和夹爪控制通常比旋转噪声更直接影响任务成功。因此，本项目在 `Flow-Matching action head` 的 `velocity prediction loss` 中加入动作维度权重。

通用 `failure-aware` 设置为：

Listing 5: 通用动作维度权重

```
[x, y, z, roll, pitch, yaw, gripper]
[1.0, 1.0, 1.0, 0.7, 0.7, 0.7, 1.5]
```

该设置强调末端位置控制和 `gripper` 开合时机，同时降低旋转维度噪声对整体 `loss` 的影响。对于 `open drawer`、`close drawer`、`lift object`、`move slider` 等任务，`gripper` 开合方向和时间点对成功率影响明显，因此该加权策略有助于提升动作执行稳定性。

6.4 Action Chunk Time Weighting

PI / LayerwiseFM 动作头一次预测一个 `action chunk`。长程闭环执行中，`chunk` 后半段动作对后续状态转移有直接影响。如果后半段动作偏差较大，模型在下次 `replan` 时会面对更偏离训练分布的状态。

因此，本项目在动作 `chunk` 的时间维度上加入线性权重，使后段动作的 `loss` 略高于前段动作。

Listing 6: 动作 chunk 时间权重

```
early-step weight: 1.0
late-step weight: 1.25
```

这鼓励模型不仅预测当前动作，还要使整个短期轨迹更加平滑和可执行，从而缓解连续任务中的误差累积。

6.5 后训练默认配置

本项目后训练采用较小学习率和较短训练步数，以避免破坏已经学到的基础操作能力。

表 9: 后训练默认配置

项目	设置
Warm-start checkpoint	【待填写】
Max train steps	5000
Action head learning rate	5e-5
Batch size per GPU	【待填写：例如 8】
Failure-aware weight	2.0
Top-K failure instructions	20
Action dim weights	[1.0, 1.0, 1.0, 0.7, 0.7, 0.7, 1.5]
Late-step weight	1.25
Frozen module	【待填写：例如 qwen_vl_interface】

冻结视觉语言 backbone 的原因主要有两点：第一，CALVIN 后训练数据量有限，解冻 VLM 容易过拟合环境 D 或破坏已有视觉语言表征；第二，冻结 VLM 可以显著降低训练成本，使后训练更适合作为快速迭代阶段。

7 Push-Aware 改进方案

7.1 问题发现

根据 per-task success rate 分析，当前模型最低成功率任务集中在 `push_{color}_block_left/right` 一类横向推动任务中。例如：

Listing 7: 低成功率 push 任务

```
push_blue_block_right: 19.4%
push_red_block_left: 24.4%
push_pink_block_left: 26.5%
push_red_block_right: 36.4%
push_pink_block_right: 41.2%
push_blue_block_left: 59.0%
```

这说明模型在 free block lateral pushing 中存在明显短板。该类任务与 `push_into_drawer` 不同，后者成功率仍达到 84.4%，说明模型不是完全不会执行 push，而是在没有轨道约束的横向推动中难以稳定控制接触和方向。

7.2 Push 任务失败根因分析

`push_{color}_block_left/right` 的低成功率可能来自以下原因：

1. 接触点不稳定：末端执行器没有准确接触 block 的合适位置。
2. 水平位移预测不足：x/y 方向动作幅度不够或方向错误。
3. 持续接触能力不足：模型在 action chunk 后半段无法保持稳定推动。
4. 颜色与方向组合泛化不足：例如 `push_blue_block_right` 明显低于其他 push 任务。
5. 长程状态偏移：前序任务造成 block 或机械臂状态变化，使后续 push 更难完成。

7.3 Push-Aware Sampling

仅依赖 failure log 中的 top-k 指令可能低估 push 的真实难度。原因是 CALVIN 评测记录的是任务链中的首次失败，如果模型在 Task 2 已经失败，则后续 push 任务可能根本没有被执行到。

因此，本项目进一步引入 Push-Aware Sampling：除 failure log 中高频失败指令外，将 push-block left/right 这类低成功率任务显式加入 hard instruction set，并提高其采样频率。

目标 hard tasks 包括：

Listing 8: Push-Aware hard tasks

```
push_red_block_left
push_red_block_right
push_blue_block_left
push_blue_block_right
push_pink_block_left
push_pink_block_right
```

该策略能够更直接地针对当前模型最明显的动作原语短板进行修复。

7.4 Push-Focused Weighted Loss

通用 failure-aware loss 强调 gripper 维度，但对于 free block pushing，主矛盾不是 gripper 开合，而是水平位移方向、幅度和持续接触。因此，本项目设计 push-focused action loss：

Listing 9: Push-focused action loss weights

```
[x, y, z, roll, pitch, yaw, gripper]
[1.6, 1.6, 0.8, 0.5, 0.5, 0.5, 0.7]
```

该权重设计的含义如下：

1. 提高 x/y 权重，强化水平推动方向和幅度。
2. 降低 z 权重，避免模型学习过多竖直动作。
3. 降低 roll/pitch/yaw 权重，减少旋转噪声对 loss 的干扰。
4. 降低 gripper 权重，使模型在 push 中更关注持续推动而不是开合动作。

7.5 两阶段 Post-Training 设计

为避免 push 专项训练损害其他任务，本项目采用两阶段后训练策略。

Stage 1: Push-Focused Repair. 该阶段集中修复 push_{color}_block_left/right 任务，采用更高的 failure-aware weight 和 push-focused loss。

表 10: Stage 1 Push-Focused Repair

项目	设置
训练步数	【待填写：例如 2000】
action learning rate	【待填写：例如 5e-5】
failure-aware weight	【待填写：例如 4.0】
top-k failure instructions	【待填写：例如 50】
action dim weights	[1.6, 1.6, 0.8, 0.5, 0.5, 0.5, 0.7]
late-step weight	【待填写：例如 1.35】

Stage 2: Mixed Consolidation. 该阶段切回更通用的 failure-aware 设置，使用较低学习率和更均衡的动作维度权重，避免模型过度偏向 push 动作而损害 lift、place、drawer 等其他任务。

表 11: Stage 2 Mixed Consolidation

项目	设置
训练步数	【待填写：例如 3000】
action learning rate	【待填写：例如 2e-5】
failure-aware weight	【待填写：例如 2.0】
top-k failure instructions	【待填写：例如 20】
action dim weights	[1.0, 1.0, 1.0, 0.7, 0.7, 0.7, 1.5]
late-step weight	【待填写：例如 1.25】

两阶段设计的动机是：Stage 1 把 push 这类明显短板修上来；Stage 2 将修复后的 push 能力重新整合回完整 CALVIN 任务分布，降低非 push 任务退化风险。如果实验结果表明 Stage 1 已经提升 push 且未损害其他任务，则 Stage 2 可以作为 optional consolidation 或 ablation 实验。

8 消融实验设计

为了验证 Failure-Aware Post-Training 和 Push-Aware 策略的有效性，本项目设计以下消融实验。

表 12: 消融实验设计

实验设置	FA	WL	Push	PL	S2	目的
Baseline finetune	否	否	否	否	否	常规 CALVIN 微调基线
Failure-Aware only	是	否	否	否	否	验证失败样本重采样效果
Weighted loss only	否	是	否	否	否	验证动作维度加权效果
FA + Weighted loss	是	是	否	否	否	验证通用后训练效果
Push-Aware Stage 1	是	是	是	是	否	验证 push 专项修复效果
Stage 1 + Stage 2	是	是	是	部分	是	验证两阶段后训练是否提升整体链式成功率

实际完成的消融实验包括：

Listing 10: 实际完成实验

【待填写：列出最终真实运行的实验】

主要观察指标包括 Task 1-5 阶段成功率、Average Chain Length、push*_block_left/right 平均成功率、bottom-k task success rate，以及非 push 任务是否出现明显退化。

9 工程实现

9.1 代码修改概述

本项目在 StarVLA 框架中主要增加了以下模块：

1. CALVIN eval failure logging。
2. failure_summary.json 自动统计。
3. failure-aware dataloader。
4. weighted Flow-Matching action loss。

5. post-training script。
6. 【待填写：如果加入 push-aware sampling，也写在这里】

9.2 Failure Logging

修改文件：

```
examples/calvin/eval_files/eval_calvin.py
```

新增功能：

1. 在 rollout 失败时记录 sequence id、subtask id、失败阶段、失败任务、完整任务链和语言指令。
2. 自动生成 failure_log.jsonl。
3. 自动生成 failure_summary.json。
4. 统计平均链长、Task 1-5 成功率、各阶段失败次数和各子任务失败次数。

9.3 Failure-Aware Dataloader

修改文件：

```
starVLA/dataloader/gr00t_lerobot/datasets.py
```

新增功能：

1. 读取 failure_log.jsonl。
2. 统计高频失败语言指令。
3. 提取 top-k hard instructions。
4. 对非 hard instruction 样本进行轻量降采样。
5. 提高失败任务在训练 batch 中的相对出现频率。

9.4 Weighted Flow-Matching Loss

修改文件：

```
starVLA/model/modules/action_model/LayerwiseFM_ActionHeader.py
```

新增功能：

1. 支持 action_loss_dim_weights。
2. 支持 action_loss_time_weights。
3. 支持 action_loss_early_step_weight 和 action_loss_late_step_weight。
4. 在 Flow-Matching velocity prediction loss 中加入维度和时间权重。

9.5 Post-Training Script

新增脚本：

```
finetunescripts/03_posttrain_failure_aware_calvin.sh
```

该脚本支持从 CALVIN finetune checkpoint warm start，并通过环境变量配置 failure log 路径、动作维度权重、时间权重、学习率和训练步数。

示例命令：

Listing 11: 启用 failure-aware 后训练

```
export PRETRAINED_CHECKPOINT=/path/to/calvin_finetune_checkpoint.pt
export FAILURE_AWARE_LOG_PATH=/path/to/failure_log.jsonl

bash finetunescripts/03_posttrain_failure_aware_calvin.sh
```

如果尚未生成 failure log，也可以只启用 weighted loss：

Listing 12: 只启用 weighted loss

```
export PRETRAINED_CHECKPOINT=/path/to/calvin_finetune_checkpoint.pt
unset FAILURE_AWARE_LOG_PATH

bash finetunescripts/03_posttrain_failure_aware_calvin.sh
```

10 可复现性说明

10.1 训练环境

表 13: 训练环境

项目	设置
操作系统	【待填写】
GPU	【待填写：例如 8 × H200】
CUDA 版本	【待填写】
Python 版本	【待填写】
PyTorch 版本	【待填写】
StarVLA 分支 / commit	【待填写】

10.2 数据路径

表 14: 数据和输出路径

项目	路径
CALVIN 数据路径	【待填写】
公开预训练数据路径	【待填写】
failure log 路径	【待填写】
输出 checkpoint 路径	【待填写】

10.3 训练命令

CALVIN 监督微调：

Listing 13: CALVIN supervised finetuning

```
export PRETRAINED_CHECKPOINT=【待填写】
export RUN_ID=【待填写】

bash finetunescripts/01_finetune_pi_calvin.sh
```

Failure-Aware Post-Training:

Listing 14: Failure-Aware Post-Training

```
export PRETRAINED_CHECKPOINT=【待填写】
export FAILURE_AWARE_LOG_PATH=【待填写】
export MAX_TRAIN_STEPS=【待填写】
export ACTION_LR=【待填写】
export FAILURE_AWARE_WEIGHT=【待填写】
export ACTION_LOSS_DIM_WEIGHTS=【待填写】
export ACTION_LOSS_LATE_STEP_WEIGHT=【待填写】

bash finetunescripts/03_posttrain_failure_aware_calvin.sh
```

CALVIN 评测:

Listing 15: CALVIN evaluation

```
python examples/calvin/eval_files/eval_calvin.py \
--args.pretrained-path 【待填写】 \
--args.unnorm-key 【待填写】 \
--args.host 【待填写】 \
--args.port 【待填写】 \
--args.dataset-path 【待填写】 \
--args.eval-sequences-path 【待填写】 \
--args.eval-log-dir 【待填写】 \
--args.num-sequences 【待填写】
```

11 实验结果分析

11.1 整体结果分析

经过【待填写: baseline / failure-aware / push-aware】后, 模型在 CALVIN ABC→D 上的平均链长从【待填写】提升到【待填写】, Task 1–Task 5 阶段成功率变化如下:

Listing 16: Task 1–5 阶段成功率变化

```
Task 1: 【待填写】
Task 2: 【待填写】
Task 3: 【待填写】
Task 4: 【待填写】
Task 5: 【待填写】
```

从结果可以看出,【待填写: 例如后训练主要提升了 Task 3–Task 5, 说明其对长程误差累积有缓解作用】。

11.2 Push 类任务结果分析

针对低成功率的 push 任务, 后训练前后结果如下:

表 15: Push 类任务前后对比

任务	Baseline 成功率	Post-Training 成功率	提升
push_blue_block_right	19.4%	【待填】	【待填】
push_red_block_left	24.4%	【待填】	【待填】
push_pink_block_left	26.5%	【待填】	【待填】
push_red_block_right	36.4%	【待填】	【待填】
push_pink_block_right	41.2%	【待填】	【待填】
push_blue_block_left	59.0%	【待填】	【待填】

如果 push-aware post-training 后上述任务成功率提升, 说明 per-task failure mining 能够有效定位模型短板, 并将评测结果转化为可操作的训练信号。如果 push 成功率提升但整体平均链长没有明显提升, 则说明模型局部动作原语得到修复, 但长程链式执行仍受其他因素影响, 例如前序任务状态偏移、视觉泛化不足或非 push 任务退化。

11.3 失败案例分析

表 16: 典型失败案例

案例	失败任务 / 阶段	失败现象	可能原因与对应改进
案例 1	【待填写】	【待填写】	【待填写】
案例 2	【待填写】	【待填写】	【待填写】
案例 3	【待填写】	【待填写】	【待填写】

12 局限性与未来改进

本项目采用的 Failure-Aware Post-Training 具有稳定、低成本、可解释的优点, 但也存在一定局限。

首先, 该方法仍然主要依赖离线监督数据, 不能像在线 RL post-training 那样直接优化真实 rollout reward。因此, 如果模型失败来自训练数据中完全没有覆盖的状态, 单纯重采样和加权 loss 的提升可能有限。

其次, 当前 failure-aware sampling 主要以语言指令作为 hard case 标识, 无法完全区分同一指令下不同初始状态导致的失败。例如, 同样是 push_blue_block_right, 不同 block 初始位置、机械臂姿态和前序任务状态可能导致不同失败原因。

第三, 如果失败主要来自环境 D 中的视觉分布变化, 仅通过 action head 后训练可能无法完全解决问题。未来可以进一步引入视觉增强、颜色扰动、域随机化或更强的跨环境数据预训练。

后续改进方向包括:

1. 引入 push-specific data augmentation, 例如 left/right 对称增强。
2. 对 failure log 记录更细粒度的状态信息, 例如失败时物体位置和机械臂末端位置。
3. 按任务类别设计不同 action loss 权重, 而不是全局统一权重。
4. 探索 replan_steps 对 push 和长程链式成功率的影响。
5. 引入轻量在线 RL 或 DAgger 式数据收集, 进一步优化真实 rollout 表现。

13 结论

本项目基于 StarVLA 框架, 在 CALVIN ABC→D 长程操控基准上实现并评测了一个 VLA Action Policy Model。通过公开数据预训练、CALVIN ABC 监督微调和 Failure-Aware Post-Training, 本项目完成了从训练、

评测、失败挖掘到针对性改进的完整实验闭环。

实验分析表明，模型在长程任务链中主要受到误差累积、动作精度不足、夹爪时机错误和环境 D 泛化失败等因素影响。其中，per-task success rate 显示 `push_{color}_block_left/right` 是当前最明显的低成功率任务簇，说明模型在 free block lateral pushing 中存在动作原语级别短板。

针对上述问题，本项目提出并实现了 Failure-Aware Post-Training，包括 failure-aware sampling、weighted action loss 和 action chunk time weighting。同时，针对 push 类任务进一步设计 Push-Aware Post-Training，通过提高 push hard instructions 的采样频率和强化 x/y 平移 loss，尝试提升模型在横向推动任务中的稳定性。

总体而言，该方法能够将 CALVIN 评测结果转化为可操作的训练信号，在较低工程风险和计算成本下，对模型长程链式执行能力进行针对性优化。

14 个人贡献说明

姓名：【待填写】

队伍：【待填写】

负责模块：【待填写】

本人在本项目中的主要贡献包括：

1. 【待填写：例如搭建 StarVLA / CALVIN 训练与评测环境】
2. 【待填写：例如完成 CALVIN ABC 监督微调脚本调试】
3. 【待填写：例如实现 failure logging 和 failure summary 统计】
4. 【待填写：例如实现 failure-aware dataloader】
5. 【待填写：例如实现 weighted Flow-Matching action loss】
6. 【待填写：例如完成 post-training 实验与结果分析】
7. 【待填写：例如整理技术文档和答辩材料】

个人贡献总结：

【待填写：用 1 段话总结个人承担的工作。建议写得具体一点，例如：
本人主要负责后训练方案设计 with 实现，包括在 CALVIN eval 中加入失败日志记录、
基于 failure_log 的 hard instruction mining、LayerwiseFM action loss 加权，
以及 push-aware post-training 实验设计。】

15 交付物链接

表 17: 交付物链接

交付物	链接 / 路径
代码仓库	【待填写：GitHub / Gitee / 内部仓库链接】
模型权重	【待填写：HuggingFace 链接或下载链接】
训练日志	【待填写】
评测日志	【待填写】
failure log	【待填写】
failure summary	【待填写】
演示视频	【待填写】
答辩 PPT	【待填写】

A 仍需补充的信息清单

最终提交前，建议优先补齐以下信息：

1. 最终选用的基座模型：Qwen3.5 几 B，还是 Qwen2.5 / Qwen3。
2. 最终 action head：PI / LayerwiseFM 是否确认。
3. 预训练 checkpoint 和 CALVIN finetune checkpoint 路径。
4. baseline 的 Task 1-5 成功率和 Average Chain Length。
5. post-training 后的 Task 1-5 成功率和 Average Chain Length。
6. push-aware 是否最终真实运行；如果运行，补充 push 类任务前后成功率。
7. 代码仓库链接、模型权重链接、个人姓名和个人贡献。

参考文献

- [1] StarVLA Team. *StarVLA: A Lego-like Codebase for Vision-Language-Action Model Developing*. arXiv:2604.05014, 2026.
- [2] Mees, O., Hermann, L., Rosete-Beas, E., and Burgard, W. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 2022.
- [3] Qwen Team. Qwen-VL and Qwen multimodal model series. Technical report and model documentation, 2024–2026.
- [4] NVIDIA. Cosmos Predict2 video world model documentation. 2026.